

Reconhecimento de dígitos manuscritos

1 Instruções

Prazo de submissão: 23:59 do dia 13/06/2026

- Antes de submeter suas soluções, certifique-se de que tudo roda como esperado. Primeiro, **reinicie o kernel** no menu, selecione Kernel → Restart e então execute **todas as células** (no menu, Cell → Run All)
- A entrega é composta por apenas um arquivo .ipynb. Ele não deve ser compactado, e só deve ter o código da seção 3 desse documento (Exercício: Reconhecimento de dígitos).
- Não deixe de preencher seu nome e número de matrícula no formato a seguir dentro do arquivo .ipynb de entrega:
Nome dos alunos e matrícula:
INSIRA Nome1: XXXX, Matricula1: XXX
INSIRA Nome2: XXXX, Matricula2: XXX

Neste documento, a parte inicial é apenas para sua orientação. Ele mostra um sistema de classificação de faces em fotos e você não precisa entregar nada acerca desta parte inicial. O que você precisa entregar é um exercício quase idêntico para classificar dígitos manuscritos usando a mesma técnica desta parte inicial. Veja a seção **Exercício** no final deste documento.

2 Exercício de Programação: PCA

2.1 Tutorial de reconhecimento de faces usando PCA - Eigenfaces

O arquivo `DadosYaleFaces.zip`, na pasta `data`, contém diretórios com arquivos de fotos de 15 indivíduos, um diretório para cada indivíduo. Dentro deles, temos 11 fotos de cada indivíduo. As fotos variam de acordo com aspectos tais como iluminação, expressão (sorrindo, sério, triste), pela presença de óculos ou não. A Figura 1 exibe uma amostra dessas fotos. Cada linha de fotos corresponde a um indivíduo. As imagens são normalizadas para alinhar os olhos e bocas. Eles aparecem mais ou menos no mesmo local na imagem. Vamos fazer uma análise das fotos via componentes principais com o objetivo de reconhecer estes rostos. Imagine que você tenha uma base de dados com várias fotos de um conjunto de indivíduos. Você vai especificar um sistema



Figure 1: Amostra de 5 fotos de 4 indivíduos

de vigilância para uma companhia e apenas os 15 indivíduos destas fotos podem entrar num certo local. A checagem é feita automaticamente com uma nova foto tirada no momento da tentativa de entrada. Vamos usar o PCA para criar um sistema para classificar esta nova foto a uma das 15 classes representadas pelos diferentes indivíduos. Assim, o problema é: Chega uma nova foto. Queremos encontrar o rosto mais parecido com a nova foto no banco de dados. Se o rosto mais parecido não estiver próximo o suficiente da nova foto, a entrada não é permitida.

O método das *autofaces* foi proposto por Turk and Pentland (1991a, 1991b). Ele é principalmente um método de redução de dimensionalidade, podendo representar muitos indivíduos com um conjunto relativamente pequeno de dados. A ideia é representar a foto de um rosto como uma soma de um rosto médio mais uma combinação linear de um pequeno número de pseudo-fotos, que são as *autofaces*. Estas *autofaces* são fotos embaçadas que capturam aspectos importantes da composição de uma face.

Podemos imaginar cada foto sendo aproximadamente obtida como representado na Figura 2. As fotos à esquerda são aproximadamente iguais a uma mesma foto média (a primeira do lado direito) mais quatro autofaces, cada uma delas multiplicada por um peso w_{ij} que é específico do indivíduo. Diferentes indivíduos vão variar apenas nos 4 pesos w_{ij} que cada autoface recebe. As autofaces são

fixas e as mesmas para todos os indivíduos considerados, bem como a face média, que também é a mesma para todos

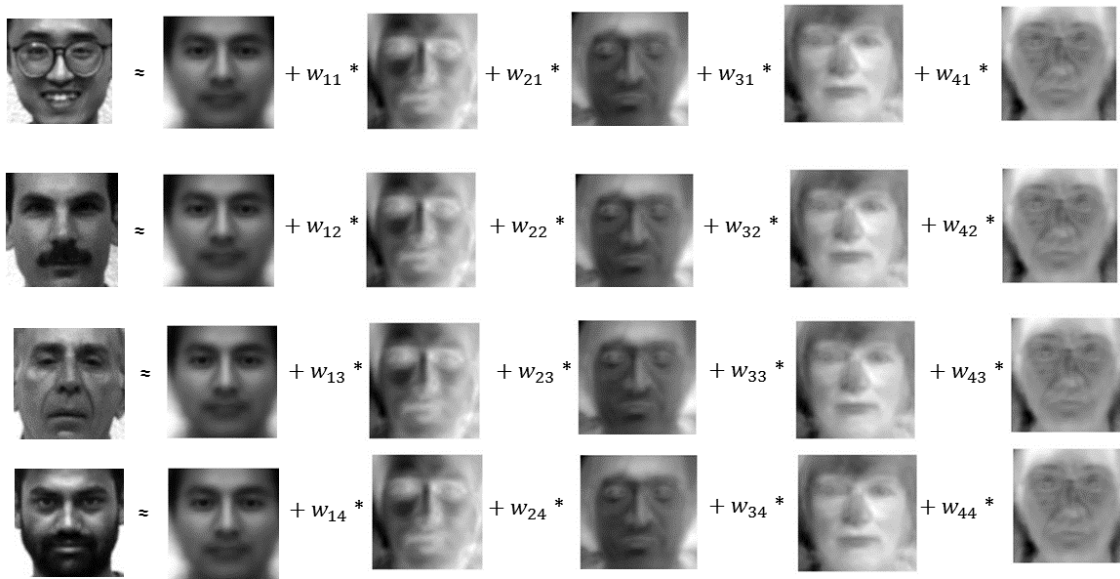


Figure 2: Uma foto (à esquerda) é aproximadamente igual à soma de uma foto média com quatro autofaces multiplicadas por pesos w_{ij} específicos do indivíduo. Diferentes indivíduos variam apenas nos quatro pesos w_{ij} atribuídos a cada autoface.

Primeiramente, vamos ler as fotos em Python. Vou usar a biblioteca Pillow que fornece algumas funcionalidades para processamento de imagens no Python. Informações em <https://pillow.readthedocs.io/en/stable/>. Essa biblioteca é baseada na biblioteca Pil (Python Image Library). O script para instalar segue abaixo:

```
[ ]: import sys
!{sys.executable} -m pip install --user Pillow scikit-learn
```

Atenção caso o script não funcione basta executar o comando a seguir

```
pip install Pillow scikit-learn
```

pode ser necessário possuir permissões de administrador.

A função `open` le imagens em um arquivo. Vamos carregar a imagem no arquivo **Faces3/s5.jpg** e visualizá-la.

```
[ ]: from PIL import Image
import matplotlib.pyplot as plt
%matplotlib inline

image = Image.open('./DadosYaleFaces/Faces3/s5.jpg')
plt.imshow(image)
```

image é um objeto do tipo Image da biblioteca Pillow. Com 100×100 pixels com 3 canais de cor, o usual RGB.

Podemos transformar o objeto image em um `numpy.array` e vice-versa. Vejam abaixo

```
[ ]: import numpy as np

#image to numpy.array
image = np.asarray(image)
print('Esse é o tamanho da imagem: Altura:',image.shape[0], ' Largura:',image.
      ↪shape[1], 'Canais de cor:',image.shape[2])

#numpy.array to image
a = np.ones((100,100,3))
a[:, :, 0] = a[:, :, 0] * 255
i = Image.fromarray(np.uint8(a))
plt.imshow(i)
```

Agora iremos ler todas as fotos dos indivíduos. Existem 15 indivíduos separados em 15 diretório. E dentro de cada um desses diretórios possuem 11 imagens em formato `jpg` e `bmp`. Cada uma dessas fotos correspondem a diferentes poses de um mesmo indivíduo. Vamos ver como fazer a leitura das fotos e armazenamento na lista `fotos`

```
[ ]: import os

fotos = []
for i in range(1,16):
    fotos_individuo = []
    for j in range(1,12):
        fotos_individuo.append(Image.open('./DadosYaleFaces/Faces'+str(i)+'/'
      ↪s'+str(j)+'.jpg'))
    fotos.append(fotos_individuo)

##Podemos exibir a imagem 7 do individuo 9, com o comando
plt.axis('off')
plt.imshow(fotos[9][7])
```

Podemos observar algumas imagens aleatórias da seguinte maneira

```
[ ]: plt.figure(figsize=(10, 10))

plt.subplot(331)
plt.imshow(fotos[1][1])
plt.subplot(332)
plt.imshow(fotos[1][2])
plt.subplot(333)
plt.imshow(fotos[1][3])

plt.subplot(334)
plt.imshow(fotos[3][1])
```

```
plt.subplot(335)
plt.imshow(fotos[3][2])
plt.subplot(336)
plt.imshow(fotos[3][3])

plt.subplot(337)
plt.imshow(fotos[8][1])
plt.subplot(338)
plt.imshow(fotos[8][2])
plt.subplot(339)
plt.imshow(fotos[8][3])

plt.show()
```

Para a análise de componentes principais, vamos converter as fotos em matrizes e colocá-las numa lista `fotosmat`

```
[ ]: fotosmat = []
for i in range(15):
    fotosmat_individuo = []
    for j in range(11):
        fotosmat_individuo.append(np.asarray(fotos[i][j]))
    fotosmat.append(fotosmat_individuo)
fotosmat = np.asarray(fotosmat)
```

Todos os canais de cor das imagens são iguais, vocês podem confirmar isso com o código abaixo.

```
[ ]: print(fotosmat[0][0][:,:,1] == fotosmat[0][0][:,:,2])
```

Agora iremos empilhar as colunas de cada uma das matrizes para montar uma matriz nova. Para isso iremos utilizar a função `reshape` passando o parâmetro `order` para utilizarmos a convenção adotada pela linguagem de programação Fortran. Mais informações <https://docs.scipy.org/doc/numpy/reference/generated/numpy.reshape.html>

Logo como todos os canais de cor são iguais iremos utilizar apenas 1 deles para realizar os cálculos.

```
[ ]: mat_pixels = []
for i in range(15):
    for j in range(11):
        mat_pixels.append(fotosmat[i][j][:,:,0].reshape((-1,1),order='F'))
mat_pixels = np.asarray(mat_pixels).T[0]
mat_pixels.shape
```

```
[ ]: import random

random.seed(42)
idx = random.randint(0,11)
print('Retiraremos a imagem de número ',idx,' de cada indivíduo.')
#Agora iremos montar a matriz de teste para o nosso método.
```

```
idxs = [idx+(i*11) for i in range(15)]
mat_teste = mat_pixels[:,idxs]
mat_pixels = mat_pixels[:,[i for i in range(165) if i not in idxs]]
```

Precisamos centrar todas as fotos do conjunto de treinamento subtraindo de cada foto a foto média de todo o conjunto de fotos. Esta foto média é simplesmente a *foto''* obtida tirando a média aritmética sobre o conjunto de fotos em cada pixel. Isto é, para um pixel localizado numa certa posição, tiramos a média de todos os valores observados naquela posição nas diferentes fotos do conjunto de treinamento.

```
[ ]: mat_media = np.mean(mat_pixels,axis=1)
mat_centrada = (mat_pixels.T - mat_media.T).T
```

Vamos visualizar como é a nossa imagem média.

```
[ ]: image = Image.fromarray(mat_media.reshape((100,100)).T)
plt.axis('off')
plt.imshow(image, cmap='gray')
```

Iremos utilizar o PCA para achar as principais componentes do nosso conjunto de dados. Iremos buscar pela quantidade de componentes que conseguem representar 90% dos dados.

```
[ ]: from sklearn.decomposition import PCA
pca = PCA(0.9)
pca.fit(mat_centrada.T)

#Quantas componentes não necessárias para representar os dados
print(pca.components_.shape)
#Podemos observar a contribuição de cada uma das componentes na
↳representatividade dos dados
print(pca.explained_variance_ratio_ )
#Somando esses valores obtemos a nossa porcentagem.
print(np.sum(pca.explained_variance_ratio_ ))

print('utilizando os 10 primeiros componentes',np.sum(pca.
↳explained_variance_ratio_[:10]))
print('utilizando os proximos 10 componentes',np.sum(pca.
↳explained_variance_ratio_[:20]))
```

Veja a variância explicada por cada um dos componentes. Os 10 primeiros PCs explicam 76% da variação total. Acrescentar mais 10, ficando com 20 PCs, leva a 86%. Vamos realizar observar melhor como a variação acrescentada por cada componente vai evoluindo com mais componentes.

```
[ ]: plt.plot(pca.explained_variance_ratio_)
plt.ylabel('Variação acrescentada')
plt.xlabel('Qtd Principais Componentes')
```

```
[ ]: autovalores = pca.explained_variance_
autovetores = pca.components_
```

Vamos criar agora as autofaces. Vamos desempilhar os autovetores quebrando a coluna de cada um deles em 100 colunas de tamanho 100 cada uma. Com isto, cada autovetor dá origem a uma *imagem* com a mesma quantidade de pixels que as fotos originais. Vamos chamar estas pseudo-fotos de **autofaces**. Iremos utilizar apenas os 20 primeiros autovetores para isso.

```
[ ]: auto_faces = []
for i in range(20):
    auto_faces.append(pca.components_[i].reshape((100,100)))
auto_faces = np.asarray(auto_faces)
#Vamos ver algumas fotos
plt.figure(figsize=(10, 10))

plt.subplot(231)
plt.imshow(auto_faces[0].T, cmap='gray')
plt.subplot(232)
plt.imshow(auto_faces[1].T, cmap='gray')
plt.subplot(233)
plt.imshow(auto_faces[2].T, cmap='gray')

plt.subplot(234)
plt.imshow(auto_faces[3].T, cmap='gray')
plt.subplot(235)
plt.imshow(auto_faces[4].T, cmap='gray')
plt.subplot(236)
plt.imshow(auto_faces[5].T, cmap='gray')

plt.show()
```

Vamos visualizar uma face em **mat_teste** sua aproximação usando as k primeiras autofaces. Para isto precisamos escrever uma foto como aproximadamente uma soma da foto média mais uma combinação linear das primeiras k autofaces (todos como vetores). Em seguida, desempilhamos os vetores e mostramos as faces. Por exemplo, usando a face na coluna 5 de **mat_teste**. Vamos obter os coeficientes b_k da combinação linear $mbox{foto} = \text{fotomedia} + b_1v_1 + b_2v_2 + \dots + b_kv_k$. Sabemos que b_j é o produto interno dos vetores formados pela foto centrada com o autovetor v_k . Então

```
[ ]: coef = autovetores@((mat_teste[:,5].T - mat_media.T).T)
```

A variável **coef** possui agora os coeficientes para a representação da foto em questão no espaço gerado pelos 20 primeiros autovetores. Vamos observar agora as aproximações para a imagem utilizando 2,3,4 e 20 autovetores.

```
[ ]: fotos_teste_5 = []
for i in [2,3,4,20]:
    vec_aproximado = mat_media + autovetores[:,i,:].T@coef[:,i]
    fotos_teste_5.append(vec_aproximado.reshape((100,100)))

plt.imshow(mat_teste[:,5].reshape((100,100)).T, cmap='gray')
```

```
plt.title('Imagem Original')

plt.figure(figsize=(10, 10))
plt.title('Aproximações')
plt.subplot(221)
plt.imshow(fotos_teste_5[0].T, cmap='gray')
plt.subplot(222)
plt.imshow(fotos_teste_5[1].T, cmap='gray')
plt.subplot(223)
plt.imshow(fotos_teste_5[2].T, cmap='gray')
plt.subplot(224)
plt.imshow(fotos_teste_5[3].T, cmap='gray')
```

Vamos realizar esse processo para todas as imagens no nosso conjunto de teste.

```
[ ]: coef_teste = autovetores[:20,:].T @ ((mat_teste.T - mat_media.T).T)
```

Como classificar cada uma destas 15 fotos em uma das categorias disponíveis, as categorias sendo os 15 indivíduos? Qual será a taxa de acerto deste sistema de classificação? Vamos primeiro obter a representação de cada uma das 150 fotos de treino nos 20 PCAs, exatamente como fizemos com as fotos de teste

```
[ ]: coef_treino = autovetores[:20,:].T @ ((mat_pixels.T - mat_media.T).T)
```

Vamos obter uma representação média de cada um dos 15 indivíduos. Vamos tirar a média dos coeficientes das 10 fotos de cada indivíduo

```
[ ]: coef_medio = []
for i in range(1,16):
    coef_medio.append(np.mean(coef_treino[:,(i-1)*10:i*10],axis=1))
coef_medio = np.asarray(coef_medio).T
```

Agora para classificar as faces do conjunto de teste entre os indivíduos devemos olhar para `coef_teste` e ver de qual `coef_medio` ele mais se aproxima.

```
[ ]: idx_proximo = []
for i in range(15):
    idx_proximo.append(np.argmin(np.mean((coef_medio -
    ↪coef_teste[i])**2,axis=1)))
print(idx_proximo)
```

2.2 Classificação perfeita!

A lista `idx_proximo` indica que a primeira imagem do conjunto de teste é relacionada ao primeiro indivíduo e assim consecutivamente.

Em conclusão, uma versão aproximada de uma foto em escala de cinza de um rosto humano pode ser obtida como uma combinação linear de umas poucas autofaces (eigenfaces, em inglês): $foto = m_{diagonal} + c_1 v_1 + \dots c_k v_k$

Os autovetores ou PCAs $v_1, \dots, v_k$ da matriz de covariância Σ da distribuição conjunta dos pixels formam as autofaces. É impressionante que apenas algumas poucas, as primeiras k , autofaces ou PCAs sejam suficientes para obter uma boa semelhança dos rostos da maioria das pessoas. As autofaces se parecem com um rosto humano médio, sem muitos traços distintivos

O método de autofaces foi proposto por Turk and Pentland (1991a, 1991b). Ele é principalmente um método de redução de dimensionalidade, podendo representar muitos indivíduos com um conjunto relativamente pequeno de dados. De fato, no nosso problema, temos uma base de 10 fotos de 15 indivíduos, cada foto armazenada como uma matriz com 10^4 elementos. Assim, temos, ao todo, $1015 \cdot 10^4 = O(10^6)$ bytes para armazenar. Vamos imaginar uma base maior com n indivíduos implicando em $O(n \cdot 10^5)$. Ao chegar uma nova foto precisamos compará-la com as $10n$ fotos por um procedimento ingênuo (naive). Com as autofaces, guardamos apenas 20 pseudo-fotos, as autofaces (possivelmente, apenas 12 seriam suficientes), e os coeficientes médios de cada indivíduo. Veja que o número de autofaces não varia muito com n . Mesmo que n seja muito grande, teremos apenas um número pequeno de autofaces. Supondo que sejam 20 autofaces, isto significa guardar $(20 \cdot 10^4) + (20n)$. Por exemplo, com $n = 1000$ e 10 fotos para cada um e com 20 autofaces teríamos que guardar e manipular $2.2 \cdot 10^5$ enquanto que uma comparação ingênua requer armazenar 10^8 , ou 1000 vezes mais espaço.

No entanto, o método de autofaces pode ter um desempenho muito ruim se existir muita diferença entre as imagens na base de treinamento e as novas imagens (Moon and Phillips, 2001). Uma imagem de um indivíduo sob iluminação frontal pode ter coeficientes muito diferentes do mesmo indivíduo, na mesma pose, sob iluminação lateral intensa. Assim, as novas fotos devem ter representantes similares na base de treino.

Autofaces é uma técnica antiga e já existem muitas variações e melhorias, bem como outras abordagens completamente diferentes para o mesmo problema. Para quem quiser se aprofundar sobre as muitas outras técnicas envolvidas com o reconhecimento de faces, visite o website <http://www.face-rec.org/general-info/>.

3 Exercício: Reconhecimento de dígitos

Este exercício é praticamente a mesma coisa que foi feita acima para as fotos. Ele foi extraído da página web do livro *The Elements of Statistical Learning*, por Hastie, Tibshirani e Friedman. Este excelente (e avançado) livro está disponível para *download* gratuito e legal na página <http://statweb.stanford.edu/~tibs/ElemStatLearn/>

O objetivo é construir um algoritmo em Python para a classificação de dígitos escritos à mão. Utilizaremos um conjunto de dados bem famoso. Chamado de *Optical Recognition of Handwritten Digits Data Set* disponível em <https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits>.

Contudo a biblioteca **sklearn** facilita a aquisição dos dados desse conjunto especificamente.

```
[ ]: import sklearn
      from sklearn import model_selection
      import matplotlib.pyplot as plt
      import numpy as np
      from sklearn.datasets import load_digits
```

```

digits = load_digits()
print('Shape do dataset:', digits.data.shape)
print('Podemos notar que ele já está no formato onde as colunas são pixels,
      ↳linearizados da imagem e cada linha é uma amostra (dígito)!')

#### Vamos visualizar alguns exemplos ####
indices = [0, 300, 500, 800, 1000, 1200, 1400, 1600] #alguns indices aleatórios
figs, axs = plt.subplots(2, 4, figsize=(8, 8))

i = 0
for ind in indices:
    axs[int(i/4)][i%4].matshow(digits.images[ind], cmap = 'gray')
    axs[int(i/4)][i%4].set_title('Classe %s\n'%(digits.target[ind]))
    i+=1

plt.show()

```

Você deve agora realizar as seguintes tarefas:

1. Separar os dados em conjunto de treino e teste (já feito).
2. Criar uma regra de classificação de novas imagens de dígitos se baseando no exemplo de Eigenfaces.
3. Use os primeiros k autovetores da matriz de covariância entre os pixels para fazer esta regra de classificação. Você deve fazer seus cálculos com $k = 5, 10, 15, 20$.
4. Usando apenas a amostra de *teste*, crie uma tabela de contingência 10x10 de confusão C . Nesta matriz C as linhas representam a classe verdadeira do dígito (de 0 a 9) e a coluna a classe em que ele foi alocado. Na entrada C_{ij} você deve colocar o número de itens (ou imagens) que caíram naquela categoria cruzada. Crie a tabela para os quatro valores distintos de $k = 5, 10, 15, 20$. Para isto, você pode usar a função *plot_confusion_matrix* disponibilizada neste notebook.
5. Calcule e imprima a precisão média e o recall médio para cada k : A precisão média é a média aritmética da precisão das 10 classes e definida como: $pm = \frac{1}{10} \sum_{i=0}^9 \frac{C_{ii}}{C_{i+}}$ com C_{i+} sendo a soma da linha i na matriz de confusão. Revocação média é a média aritmética da revocação das 10 classes e definida como: $rm = \frac{1}{10} \sum_{i=0}^9 \frac{C_{ii}}{C_{+i}}$ com C_{+i} sendo a soma da coluna i na matriz de confusão. Você pode usar a função `sklearn.metrics.confusion_matrix()` para gerar o array da matriz de confusão. Mais detalhes sobre precisão (precision) e revocação (recall) podem ser vistos no verbete Precision and recall na wikipedia. Ver também <http://www.text-analytics101.com/2014/10/computing-precision-and-recall-for.html>.

```

[ ]: #PARTE 1 - Separando em conjunto de treino e teste

X = digits.data
y = digits.target
X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y,
      ↳test_size=0.1, random_state=1)

```

```

#Confirmando que temos todas as classes no conjunto de teste
y_aux = np.argsort(y_test)
y_test = y_test[y_aux]
X_test = X_test[y_aux]
items, counts = np.unique(y_test, return_counts=True)

print('Itens: ', items)
print('Número de vezes em que o item aparece:', counts)

```

```

[ ]: # Funcao para plotar a matriz de confusão - Você pode usá-la para visualizar o
↳ desempenho do seu classificador.
# confira um exemplo de uso no final desta célula

from sklearn.metrics import confusion_matrix
from sklearn.utils.multiclass import unique_labels

def plot_confusion_matrix(y_true, y_pred, classes,
                           normalize=False,
                           title=None,
                           cmap=plt.cm.Blues):
    """
    This function prints and plots the confusion matrix.
    Normalization can be applied by setting `normalize=True`.
    """
    if not title:
        if normalize:
            title = 'Normalized confusion matrix'
        else:
            title = 'Confusion matrix, without normalization'

    # Compute confusion matrix
    cm = confusion_matrix(y_true, y_pred)
    # Only use the labels that appear in the data
    classes = np.array(classes)
    classes = classes[unique_labels(y_true, y_pred)]

    fig, ax = plt.subplots(figsize = (4,4))
    im = ax.imshow(cm, interpolation='nearest', cmap=cmap)
    ax.figure.colorbar(im, ax=ax)
    # We want to show all ticks...
    ax.set(xticks=np.arange(cm.shape[1]),
           yticks=np.arange(cm.shape[0]),
           # ... and label them with the respective list entries
           xticklabels=classes, yticklabels=classes,
           title=title,
           ylabel='True label',

```

```

        xlabel='Predicted label')

ax.set_ylim(cm.shape[0] -1 + 0.5, -0.5)

# Rotate the tick labels and set their alignment.
plt.setp(ax.get_xticklabels(), rotation=45, ha="right",
        rotation_mode="anchor")

# Loop over data dimensions and create text annotations.
fmt = '.2f' if normalize else 'd'
thresh = cm.max() / 2.
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        ax.text(j, i, format(cm[i, j], fmt),
                ha="center", va="center",
                color="white" if cm[i, j] > thresh else "black")

fig.tight_layout()
plt.show()

##### Exemplo de como usar a função:
→#####

y_true = [0,0,3,5,2,1,3,4,5,2,1,3] # Aqui são os valores verdadeiros dos dígitos
y_pred = [0,0,3,4,3,2,3,4,5,2,0,2] # Aqui são os valores preditos pelo seu
    → método (que vão ser gerados por ele, mas neste exemplo estamos colocando na
    → mão)
classes = [0,1,2,3,4,5] # Classes possíveis: nesse exemplo de 0 a 5

#Aqui só obtemos o numpy array da matriz e exibimos (útil para o exercício 5)
cm_array = confusion_matrix(y_true, y_pred)
print(cm_array)

#Aqui plotamos a matriz em um formato bonito :) (útil para o exercício 4)
plot_confusion_matrix(y_true, y_pred, classes, title = "Matriz de confusão de"
    → exemplo:")

```